

企业知识库系统技术白皮书

一、系统概述

本系统是一套面向企业的智能知识库管理平台，采用检索增强生成（RAG）架构，结合语义搜索与知识图谱技术，实现对海量非结构化文档的智能解析、存储与精准问答。系统支持 PDF、Word、Markdown 等多种格式的文档导入，能够自动识别文档结构并构建语义索引。

系统的核心设计目标包括：第一，提供亚秒级的文档检索响应时间；第二，支持百万级文档的横向扩展能力；第三，确保数据全链路不出企业内网，满足合规要求；第四，通过多轮自适应检索机制，将问答准确率维持在 85% 以上。

二、技术架构

2.1 整体架构

系统采用分层微服务架构，自上而下分为接入层、业务层、引擎层和数据层四个逻辑层次。接入层负责 HTTP 请求的接收、认证、限流和路由分发，基于 FastAPI 框架构建，部署在 Uvicorn ASGI 服务器之上。业务层编排文档处理的完整流水线——从文档上传、格式检测、解析路由到分块策略的选择和知识图谱的增量更新。

引擎层是系统的核心，包含六个独立的功能模块：文档解析引擎负责将不同格式的源文件转换为统一的中间表示（UIR）；语义分块引擎基于句子嵌入模型的相似度计算，在文档的主题边界处进行动态切分；向量嵌入引擎使用 BGE 中文预训练模型将文本片段编码为 512 维浮点向量；向量检索引擎基于余弦相似度实现高效的近似最近邻搜索；知识图谱引擎通过规则匹配与大语言模型协作完成实体识别和关系抽取；重排序引擎利用 Cross-Encoder 模型对初步检索结果进行精细排序。

数据层采用异构存储架构：向量数据存储在 LanceDB 嵌入式数据库中（生产环境可切换为 Milvus 集群），知识图谱数据存储在 Neo4j 图数据库中，文档元数据存储在 PostgreSQL 关系型数据库中，缓存与会话数据存储在 Redis 内存数据库中。

2.2 技术选型原则

每一项技术选型均遵循以下原则：第一，优先选择国产或开源方案，避免供应商锁定；第二，开发环境追求零配置与快速启动，生产环境追求高可用与水平扩展；第三，所有外部依赖必须具备至少一级降级策略；第四，优先选择在中文场景有充分验证的模型和工具。

三、文档解析引擎

3.1 多引擎协同解析架构

文档解析是知识库系统的基础环节，其质量直接影响后续分块、检索和问答的效果。系统设计了分层的三引擎解析架构，根据输入文档的类型自动路由到最优的解析器组合。

对于原生 PDF 文档（即由 Word、LaTeX 等软件直接导出的 PDF），系统优先使用 PyMuPDF 引擎进行文本提取。PyMuPDF 基于 MuPDF C 语言库构建，在文本提取速度上比纯 Python 方案快 5 至 10 倍，且对中文字符的兼容性经过了充分验证。对于包含复杂表格的 PDF 文档，系统会同时启动 PDFPlumber 引擎进行表格区域的坐标级精确解析。PDFPlumber 通过读取 PDF 的底层绘图指令流，能够准确还原单元格的跨行跨列关系，其表格提取精度在标准测试集上达到 97% 以上。

对于扫描件或图片型 PDF，系统调用 PaddleOCR 引擎进行光学字符识别。PaddleOCR 是百度飞桨团队开发的中文 OCR 引擎，在简体中文识别准确率上达到 98.7%，优于开源的 Tesseract 引擎（中文准确率约 88%），且支持离线部署。

3.2 统一中间表示 (UIR)

为了解决不同解析器输出格式不一致的问题，系统定义了一套统一中间表示 (Unified Intermediate Representation, UIR)。无论底层使用哪种解析引擎，最终都会将解析结果标准化为 UIR 格式。UIR 的核心数据结构包括页面列表 (pages)、文本块列表 (text_blocks)、表格列表 (tables) 和图片列表 (images)。每个文本块记录其所在的页面坐标、字体大小、是否为标题等元信息，为后续的语义分块提供结构线索。

四、语义分块策略

4.1 基于标题树与语义相似度的动态分块

传统的文档分块方法——如 LangChain 的 RecursiveCharacterTextSplitter——按照固定的字符数量进行切分，完全忽略了文档的语义结构。这种方式经常导致段落在中间被截断，甚至将一句话切成两半，严重影响后续的检索质量。

本系统采用了一种结合文档结构感知与语义相似度的动态分块策略。首先，利用 PDF 解析阶段提取的字体大小和位置信息构建标题树，识别出文档的层级结构（一级标题、二级标题等）。每个文本块继承其所在的标题链，例如「第一章 > 1.1 系统概述 > 1.1.1 架构设计」。标题链提供了重要的上下文信息，使得检索时不仅能够匹配文本内容，还能利用文档结构进行精确导航。

其次，系统使用 BGE 嵌入模型将每个候选句子编码为向量，计算相邻句子之间的余弦相似度。当相似度低于自适应阈值（当前窗口相似度均值减去 0.5 倍标准差）时，系统判定此处为主题切换点并进行切分。这种自适应阈值的设计使得分块策略能够适应不同密度和风格的文档——技术白皮书通常段落紧密、相似度高，而产品手册则频繁切换主题。

分块的参数经过实验调优：最小令牌数为

100（低于此值的片段缺乏足够的上下文信息），最大令牌数为 800（接近 BGE

模型的最优输入窗口 512 令牌），硬切重叠率为 10%（避免关键信息出现在两个 chunk 的边缘而丢失）。

4.2 分块质量评估

在包含 50 份中文技术文档的测试集上，语义分块策略相比固定大小分块策略在检索召回率（Recall @10）上提升约 12 个百分点，在答案准确率上提升约 8 个百分点。特别是在包含多级标题和混合内容的复杂文档中，结构感知带来的提升更为显著。

五、向量嵌入与检索

5.1 嵌入模型选择

系统选用北京智源人工智能研究院 (BAAI) 开发的 BGE-small-zh-v1.5 作为文本嵌入模型。BGE 系列模型在 C-MTEB (中文大规模文本嵌入基准评测) 中综合排名前列, 其中文语义区分度经过了大规模中文语料的充分训练。

选择 512 维的 small 版本而非 768 维的 base 版本或 1024 维的 large 版本, 基于以下工程考量: 第一, 精度差异在中文场景下通常不超过 3%, 但 small 版本的模型体积仅为 100MB, base 版本为 400MB, large 版本超过 1GB; 第二, CPU 推理延迟分别为 20ms、45ms 和 90ms 每条文 本, 在无 GPU 的生产环境中, 延迟差异将成倍放大; 第三, 512 维向量在 LanceDB 中存储和检索的开销显著低于更高维度。

需要注意的是, BGE 模型在训练时使用了 passage: 和 query: 前缀来区分段落编码和查询编码。实测表明, 如果不加此前缀, 检索精度会下降约 5 个百分点。系统在 EmbeddingService 中自动为段落文本添加 passage: 前缀, 为查询文本添加 query: 前缀。

5.2 向量存储方案

在开发环境中, 系统使用 LanceDB 作为嵌入式向量数据库。LanceDB 采用与 Apache Arrow 深度集成的列式存储格式, 支持多版本并发控制 (MVCC), 读写操作不互锁。与广泛使用的 Chroma 相比, LanceDB 在 10 万级向量的检索场景中延迟低约 40%, 且内存占用更少。

生产环境中推荐将向量存储升级为 Milvus 分布式向量数据库。Milvus 支持 IVF (倒排文件索引) 和 HNSW (分层可导航小世界图) 等多种近似最近邻索引算法, 具备水平扩展能力和 GPU 加速推理, 并且原生集成了 Prometheus 监控端点。切换方案通过 VectorStore 的抽象接口实现, 业务代码无需修改。

六、知识图谱构建

6.1 规则与 LLM 混合抽取

知识图谱是提升检索深度和广度的关键组件。系统采用规则匹配与大语言模型相结合的混合抽取策略。对于常见的计算机技术实体——包括编程语言、框架、数据库、协议、文件格式等——系统使用预定义的 TECH_PATTERNS 正则表达式集合进行抽取。规则抽取的准确率接近 100%，且零延迟、零 API 调用成本。

对于非技术领域的实体——如公司名称、产品名称、法规编号、行业术语——系统调用大语言模型从上下文中进行抽取。LLM 同时也负责识别实体之间的关系。系统定义了 8 种标准关系类型：uses（使用）、depends_on（依赖）、contains（包含）、supplies（提供）、references（引用）、employs（雇佣）、owns（拥有）以及 signs（签署）。通过将关系类型限定在这 8 种以内，既覆盖了企业文档的常见关联模式，又避免了大语言模型生成不一致或不可控的关系类型。

6.2 图谱检索与多跳推理

在问答环节，系统对用户问题中的实体进行识别，然后在知识图谱中执行多跳遍历查询。例如，用户提问「FastAPI 依赖哪些底层组件」，系统会首先定位 FastAPI 实体节点，然后沿 depends_on 关系进行多跳扩展，发现 FastAPI 依赖 Starlette 和 Pydantic，Starlette 又依赖 Uvicorn 和 anyio，最终形成完整的依赖链路返回给用户。

多跳推理的深度限制为 3 跳，这是工程实践中在检索广度和响应延迟之间的平衡点。超过 3 跳的图遍历不仅导致延迟显著增加，而且引入的实体与原始问题的语义相关性也大幅下降。

七、混合检索与 Self-Retrieval

7.1 三路召回与精排

系统的检索流程分为召回和排序两个阶段。在召回阶段，系统同时启动三路检索：第一路为稠密向量检索，通过计算查询嵌入与文档嵌入的余弦相似度，从 LanceDB 中召回 Top-40 的候选文档；第二路为知识图谱检索，识别查询中的实体并在图谱中执行多跳遍历，召回相关的文档节点；第三路为关键词检索（基于 jieba 分词），用于补充向量检索可能遗漏的精确术语匹配。

三路召回的结果按照 chunk_id 进行合并去重，然后进入排序阶段。排序阶段使用 BGE-Reranker-base 模型（Cross-Encoder 架构）对候选项进行精细排序。Cross-Encoder 与 Bi-Encoder 的核心区别在于：Bi-Encoder（如 BGE Embedding）分别编码查询和文档，然后计算余弦相似度；Cross-Encoder 则将查询和文档拼接在一起输入模型，让模型直接对两者之间的语义交互进行建模。实测表明，Cross-Encoder 相比 Bi-Encoder 的 MRR 指标提升约 5 至 15 个百分点。

7.2 Self-Retrieval 自适应检索

传统 RAG 系统的一次性检索策略存在明显缺陷：如果首次检索的结果质量不高，系统没有纠正的机会。Self-Retrieval 机制通过在检索之后增加评估、改写和重检的闭环来解决这个问题。

评估器从三个维度衡量检索质量：相关性（relevance，检索结果与查询的语义匹配度）、覆盖率（coverage，检索结果是否覆盖了问题的不同方面）和置信度（confidence，评分分布的集中程度）。当评估器判定检索质量不足时——例如 $\text{relevance} < 0.5$ 或 $\text{coverage} < 0.5$ ——系统会根据评估反馈选择相应的改写策略：关键词扩展（keyword_expand）、同义词替换（synonym）、问题分解（decompose）或摘要调整（abstract_adjust）。改写后的查询重新进入检索流水线，形成多轮闭环。

系统将最大重试轮数限制为 3 轮。工程实践表明，超过 3 轮的改写已难以带来实质性的检索质量提升，反而导致用户感知的响应延迟成倍增长。在典型的业务场景中，超过 80% 的查询在第一轮即可获得满意的检索结果，约 15% 的查询需要第二轮改写，仅约 5% 的查询需要进入第三轮。

八、部署与运维

8.1 部署拓扑

生产环境的推荐部署拓扑包含以下组件：至少 2 个 API 服务实例（通过 Nginx 反向代理进行负载均衡）、至少 2 个 Celery Worker 实例（负责异步文档处理）、1 个 Milvus 集群（3 节点，负责向量存储与检索）、1 个 Neo4j 单机实例（或集群，负责知识图谱存储）、1 个 Redis 实例（负责限流计数、缓存和 Celery 消息代理）、1 个 PostgreSQL 实例（负责文档元数据持久化）以及 1 个 MinIO 对象存储服务（负责原始文档持久化）。

8.2 监控与告警

系统通过 Prometheus 采集关键性能指标，通过 Grafana 进行可视化展示。核心监控指标包括：QA 请求速率（按检索模式和成功/失败状态分组）、QA 响应延迟分布（P50/P95/P99）、LLM Token 消耗速率（按 prompt/completion 分组）、检索轮数分布（反映 Self-Retrieval 的触发频率）、向量库规模、知识图谱规模以及 LLM 调用错误分布（按错误类型分组）。

告警规则覆盖以下场景：QA 错误率超过 5% 持续 5 分钟、QA P99 延迟超过 10 秒持续 5 分钟、LLM API 调用成功率低于 95% 持续 5 分钟、向量库可用存储低于 80% 以及 Celery Worker 存活数低于 2。

九、性能基准

以下数据基于 16GB RAM、8 核 CPU、无 GPU 的开发环境实测（测试文档集：100 份中文技术 PDF，平均每份 8 页，生成约 4500 个 chunks）：

文档解析吞吐量为每页 0.3 秒（PyMuPDF 原生 PDF）至 2.1 秒（PaddleOCR 扫描件）；语义分块速度为每 1000 字符 0.05 秒；向量嵌入速度为每 chunk 0.02 秒（CPU 推理）；向量检索延迟为 12ms（千级向量）至 180ms（十万级向量）；QA 端到端延迟中位数为 1.2 秒，P99 为 3.8 秒；单次 QA 的 LLM Token 消耗均值约 1200 tokens（其中 prompt 约 900 tokens，completion 约 300 tokens）。